

UNIVERSIDAD CATOLICA DE HONDURAS

NUESTRA SEÑORA REINA DE LA PAZ



ASIGNACIÓN:

Documentación de proyecto- Inventario

ASIGNATURA:

Seminario-Taller de Software

CATEDRÁTICO:

Jarvin Calderón

INTEGRANTES:

Yessenia Nicolle Baquedano Cruz 0601200502457

Dania Licet Montes Carcamo 180720031818

Annyie Jaqueline Carcamo Vallecillo 07032002015641

Gladys Lizeth Salmerón Ordoñez 0703200004979

Edgar Muñoz Fuentes 1709200301066

Lidia Valeria Guifarro Escobar 1503200602463

Índice

1. Introducción.....	3
2. Planteamiento del Problema.....	4
3. Análisis y Requisitos del Sistema.....	4
3.1 Requerimientos Funcionales.....	4
3.2 Requerimientos No Funcionales.....	6
4. Justificación del Stack Tecnológico.....	6
5. Arquitectura y Estructura del Sistema.....	8
5.1 Patrón Arquitectónico MVC.....	8
5.2 Flujo de una Petición HTTP.....	9
5.3 Estructura de Archivos del Proyecto.....	10
6. Principales Módulos y Componentes del Sistema.....	10
6.1 Sistema de Roles y Control de Acceso.....	12
6.2 Algoritmo PEPS / FIFO en Cascada.....	12
7. Estructura de la Base de Datos.....	13
8. Requisitos Mínimos para Ejecutar el Sistema.....	15
8.1 Instrucciones de Instalación.....	16
9. Limitaciones Actuales del Proyecto.....	16
10. Posibles Mejoras Futuras.....	17
Conclusiones.....	19

1. Introducción

Las microempresas hondureñas —pulperías, ferreterías y pequeños comercios— administran su inventario de forma manual mediante libretas, hojas de cálculo sin estructura y estimaciones visuales. Este enfoque provoca pérdidas directas por robo hormiga, mermas no registradas, ruptura de stock inesperada y decisiones de compra basadas en intuición más que en datos confiables.

El presente informe documenta técnicamente el Sistema de Control de Inventarios desarrollado por el equipo de proyecto, una aplicación web construida sobre PHP con arquitectura MVC propia. Su propósito es automatizar el registro de productos, categorías, entradas, salidas y mermas, permitiendo al propietario tomar decisiones oportunas con información confiable y en tiempo real, sin necesidad de invertir en infraestructura costosa.

El sistema implementa la metodología PEPS (Primero en Entrar, Primero en Salir) a través de un control de lotes por compra, garantizando la trazabilidad contable de cada unidad de mercancía desde su ingreso hasta su salida del inventario.

2. Planteamiento del Problema

La gestión empírica del inventario representa uno de los principales focos de pérdida económica para las microempresas en Honduras. En ausencia de un sistema formal, los propietarios no cuentan con información precisa sobre el estado real de sus existencias, los costos de adquisición por lote ni los márgenes de ganancia efectivos.

Entre las consecuencias más comunes de esta problemática se identifican: el robo hormiga difícil de detectar sin registros de movimientos, el vencimiento de productos por falta de control de fechas, las compras innecesarias o tardías por desconocer el stock mínimo, y la imposibilidad de calcular con precisión el valor monetario del inventario total.

La solución propuesta es un sistema de información accesible, de bajo costo de infraestructura y orientado al perfil tecnológico real de las microempresas hondureñas, que digitalice y automatice estos controles críticos de inventario.

3. Análisis y Requisitos del Sistema

3.1 Requerimientos Funcionales

Los requerimientos funcionales definen las capacidades operativas que el sistema debe proveer a sus usuarios. La siguiente tabla resume los diez requerimientos funcionales identificados durante el análisis:

Requerimiento Funcional	Criterio de Aceptación
Gestión de Categorías de Productos (CRUD)	El administrador puede crear, editar y desactivar categorías.

Requerimiento Funcional	Criterio de Aceptación
Gestión de Productos (CRUD) con código interno y de barras	Cada producto tiene precio de venta, costo, stock mínimo y estado.
Registro de Entradas de Inventario (compras / recepciones)	Genera movimiento tipo ENT y actualiza stock automáticamente.
Registro de Salidas de Inventario (ventas / consumos)	Genera movimiento tipo SAL y descuenta stock. Impide negativo.
Registro de Mermas y Pérdidas	Genera movimiento tipo MER con motivo obligatorio.
Historial de Movimientos (Kardex)	Vista paginada con filtros por producto, tipo y rango de fechas.
Control de Lotes e identificación de vencimiento (PEPS)	El sistema lleva lotes con fecha de ingreso y vencimiento opcional.
Alerta de Stock Bajo	Se despliega aviso cuando el stock actual cae por debajo del mínimo.
Autenticación de Usuarios y control de sesión	Login con correo y contraseña bcrypt. Cierre de sesión seguro.
Control de Acceso por Roles (PRP, ADM, AUD)	Cada rol tiene funciones asignadas. El auditor solo tiene acceso de lectura.

3.2 Requerimientos No Funcionales

Además de las funcionalidades explícitas, el sistema debe cumplir con los siguientes atributos de calidad:

- Rendimiento: El sistema debe responder en menos de 2 segundos para consultas habituales de inventario.
- Seguridad: Contraseñas almacenadas con bcrypt. Sesiones con token de servidor. Consultas con PDO y prepared statements para prevenir inyección SQL.
- Usabilidad: Interfaz responsive (CSS grid propio), navegable desde dispositivos móviles.
- Mantenibilidad: Arquitectura MVC con separación clara de Controladores, Vistas (.tpl) y DAO con PSR-0 autoloading vía Composer.
- Escalabilidad: La base de datos soporta la incorporación futura de módulos de facturación (campos refTipo / refId ya presentes en movimientos_inventario).
- Compatibilidad: PHP 7.2+ y MySQL 5.7+ o MariaDB 10.3+. Funcional en Apache/Nginx sobre XAMPP, WAMP o servidor Linux.

4. Justificación del Stack Tecnológico

La selección de tecnologías del proyecto priorizó tres criterios fundamentales: bajo costo de infraestructura, familiaridad del equipo con las herramientas y disponibilidad local de servicios de hosting en Honduras. La siguiente tabla presenta las tecnologías elegidas, las alternativas evaluadas y la justificación técnica de cada decisión:

Capa	Tecnología Elegida	Alternativas	Razón de Elección
Backend	PHP 7.4+ (MVC propio)	Node.js, Python/Django	Dominio del equipo; framework propio probado; sin dependencias de runtime adicionales.
Base de Datos	MySQL / MariaDB	PostgreSQL, SQLite	Disponible en todos los hosting compartidos de Honduras. Amplia documentación con phpMyAdmin.
Frontend	HTML5 + CSS Grid (compilado desde LESS)	Bootstrap, Tailwind	CSS propio evita dependencias CDN innecesarias; LESS permite variables y mixins sin build tool pesado.
Motor de Plantillas	SimplePHPMvcOOP (.tpl)	Twig, Blade, Smarty	Integrado al framework; sintaxis {{variable}} simple; sin dependencias adicionales.

Capa	Tecnología Elegida	Alternativas	Razón de Elección
Dependencias	Composer	Instalación manual	Estándar de la industria PHP. Maneja autoloading PSR-0 y paquetes adicionales.
Servidor Web	Apache (XAMPP / cPanel)	Nginx, Caddy	Entorno estándar en laboratorios universitarios y hosting económico en Honduras.

La combinación de estas tecnologías garantiza que el sistema pueda desplegarse en cualquier hosting compartido de cPanel —el tipo de hosting más económico y disponible en el mercado local hondureño— sin requerir configuraciones especiales de servidor ni licencias de software propietario.

5. Arquitectura y Estructura del Sistema

5.1 Patrón Arquitectónico MVC

El sistema implementa el patrón arquitectónico Modelo – Vista – Controlador (MVC) de forma artesanal en PHP, sin dependencia de megaframeworks externos. Cada capa tiene una responsabilidad única y una comunicación estrictamente unidireccional, lo que facilita el mantenimiento y la incorporación de nuevos módulos sin afectar el resto del sistema.

La capa de Router o Bootstrap, ubicada en `index.php` y `autoloader.php`, recibe cada petición HTTP, carga el autoloader PSR-0 de Composer y despacha la solicitud al controlador correspondiente según el parámetro `?page=` recibido en la URL.

La capa de Controlador, ubicada en `src/Controllers/`, valida los permisos del usuario activo a través de `PrivateController`, procesa la lógica de negocio, invoca al DAO correspondiente y construye el arreglo de datos `$viewData` que será enviado al motor de plantillas.

La capa de DAO / Modelo, ubicada en `src/Dao/`, hereda de `Table.php` para obtener una conexión PDO singleton. Contiene todas las consultas SQL parametrizadas de cada entidad del sistema, garantizando la prevención de inyección SQL mediante `prepared statements`.

La capa de Vista, ubicada en `src/Views/templates/`, contiene los archivos `.view.tpl` con sintaxis `{{variable}}`, `{{foreach}}` y `{{if}}`. El `Renderer` combina estas plantillas con el layout general y reemplaza todas las variables dinámicas antes de enviar el HTML final al navegador.

Las Utilidades y Seguridad, ubicadas en `src/Utilities/`, albergan las clases `Security` (manejo de sesión, `bcrypt` y roles), `Nav` (generación dinámica del menú lateral según rol), `Context` (variables globales del sistema) y `Dao.php` (singleton de conexión PDO).

5.2 Flujo de una Petición HTTP

El siguiente flujo describe el ciclo completo de procesamiento de una solicitud dentro del sistema, tomando como ejemplo el acceso al módulo de categorías:

- El navegador envía: `GET index.php?page=mnt_categorias`
- `index.php` carga `autoloader.php` (PSR-0 Composer) y lee el archivo `parameters.env` con las credenciales de base de datos.
- Se instancia `Controllers\Mnt\Categorias`; el constructor de `PrivateController` verifica sesión activa y autorización por rol.
- El método `run()` consulta al DAO (`Dao\Mnt\Categorias::obtener()`) y construye el arreglo `$viewData` con los datos necesarios para la vista.
- `Renderer::render('mnt/categorias', $viewData)` carga `privatelayout.view.tpl`, inyecta el contenido del template de página y reemplaza todas las variables `{{...}}`.

- El HTML final compilado se envía al navegador del cliente.

5.3 Estructura de Archivos del Proyecto

La organización de carpetas sigue el patrón MVC con separación estricta de responsabilidades. A continuación se describen los directorios y archivos más relevantes:

- proyecto-inventario-master/ — Raíz del proyecto
- index.php — Punto de entrada único (Front Controller)
- autoloader.php — Registro del autoloader PSR-0
- composer.json — Definición de dependencias del proyecto
- renameTo_parameters.env — Plantilla de configuración del entorno
- src/Controllers/ — Controladores MVC organizados en subcarpetas (Admin, Mnt, Sec)
- src/Dao/ — Objetos de Acceso a Datos que heredan de Table.php y Dao.php
- src/Utilities/ — Clases auxiliares: Security, Nav, Context
- src/Views/templates/ — Plantillas HTML (.view.tpl) y layouts del sistema
- public/css/ — Hojas de estilo compiladas desde archivos LESS
- docs/scripts/ — Scripts SQL de creación de tablas y datos iniciales

6. Principales Módulos y Componentes del Sistema

El sistema está organizado en módulos funcionales claramente delimitados. Cada módulo agrupa un conjunto de controladores, DAOs y plantillas de vista que colaboran para cubrir un área específica del dominio del negocio:

Módulo	Componentes Principales	Función
Seguridad (Sec)	Controllers/Sec/, Dao/Security/, templates/security/	Login, cierre de sesión.
Administración (Admin)	Controllers/Admin/, templates/admin/	Dashboard resumen.
Mantenimiento (Mnt)	Controllers/Mnt/, templates/mnt/	CRUD de Categorías, Productos y gestión de Usuarios del sistema.
Inventario – Kardex	Controllers/Mnt/Kardex.php, Dao/Inventario	Registro y consulta del historial de movimientos ENT/SAL/MER con trazabilidad por lotes.
Proveedores	Controllers/Mnt/Proveedores. php	CRUD de proveedores e integración al formulario de creación de productos.
Base de Datos	docs/scripts/*.sql	Scripts de creación de tablas: usuario, roles, categorías, productos, lotes, movimientos.
Assets / Presentación	public/css, public/imgs	Estilos responsive compilados desde LESS, iconografía Font Awesome y recursos estáticos.

6.1 Sistema de Roles y Control de Acceso

El control de acceso es una de las características más críticas del sistema. Se implementan tres niveles de acceso predefinidos:

- Propietario / Superusuario (PRP): Acceso total para leer, insertar, editar y eliminar en todos los módulos del sistema. Es el único rol que puede gestionar usuarios y configuraciones de seguridad.
- Administrador / Empleado (ADM): Permiso para operaciones cotidianas de inventario: registro de productos, categorías, entradas y salidas. No tiene acceso a cambiar roles ni configuraciones de seguridad.
- Auditor (AUD): Acceso estrictamente de solo lectura. El sistema oculta automáticamente los botones de acción en el frontend y los controladores rechazan cualquier petición POST de modificación.

6.2 Algoritmo PEPS / FIFO en Cascada

La gestión de inventario bajo la metodología PEPS (Primero en Entrar, Primero en Salir) constituye el núcleo contable del sistema. Al registrar una salida o merma, el usuario puede optar por dos comportamientos:

- Selección manual de lote: El sistema proporciona un buscador de autocompletado en tiempo real que permite al usuario localizar un lote activo específico y descontar las unidades directamente de él.
- Deducción automática en cascada: Si no se selecciona un lote específico, el sistema ejecuta una consulta transaccional que identifica los lotes activos más antiguos (ordenados por fecha de vencimiento o fecha de ingreso) y consume su stock secuencialmente hasta cubrir la cantidad total del ajuste registrado.

7. Estructura de la Base de Datos

El modelo de datos relacional del sistema está compuesto por nueve tablas principales que mantienen integridad referencial mediante llaves foráneas. El script principal de instalación es docs/scripts/00_init_inventario.sql, el cual incluye la creación completa de todas las tablas, roles iniciales, usuarios de prueba y categorías semilla:

Tabla	Campos Clave	Propósito
usuario	usercod (PK), useremail, userpswd, userest, usertipo	Registro de usuarios del sistema con contraseña cifrada con bcrypt.
roles / roles_usuarios	roles: rolescodb (PK - varchar), rolesdsc (descripción - varchar) y rolesest (estado - char). roles_usuarios: usercod (FK - bigint) y rolescodb (FK - varchar).	Catálogo de roles (PRP, ADM, AUD) y su asignación temporal a usuarios.
funciones / funciones_rol	funciones: fncod (PK), fnac, fnest, fntyp. funciones_rol: rolescodb (FK) y fncod(FK)	Control granular de acceso a pantallas y controladores por rol.

Tabla	Campos Clave	Propósito
categorias	catid (PK), catnom, catest	Clasificación de productos. Permite filtrar y agrupar artículos del inventario.
proveedores	provId (PK), provNombre, provContacto, provTelefono, provEmail	Datos de contacto de los distribuidores asociados a productos.
productos	invPrdId (PK), invPrdBrCod, invPrdCodInt, invPrdDsc, catid (FK), provId (FK), invPrdStock, invPrdStockMin	Catálogo principal de artículos con precios, costos, stock actual y mínimo.
lotes_inventario	loteId (PK), invPrdId (FK), loteCod, loteCantActual, loteFechaVencimiento, loteCostoUnitario	Control de lotes por compra física para soporte del algoritmo PEPS/FIFO.
movimientos_inventario	movId (PK), invPrdId (FK), loteId (FK), movTipo (ENT/SAL/MER), movCantidad, movMotivo	Kardex contable con historial cronológico completo de ajustes de stock.

8. Requisitos Mínimos para Ejecutar el Sistema

Para instalar y ejecutar el sistema en un entorno local o de producción compartido, se requieren los siguientes componentes de software en sus versiones mínimas especificadas:

Componente	Requisito / Versión Mínima
Lenguaje de Servidor	PHP 7.2 o superior (probado hasta PHP 8.2)
Motor de Base de Datos	MySQL 5.7+ o MariaDB 10.3+
Servidor Web	Apache 2.4+ con mod_rewrite habilitado
Gestión de Dependencias	Composer 2.x (incluido composer.phar en el proyecto)
Extensiones PHP requeridas	pdo, pdo_mysql, mbstring, openssl, curl
Entorno recomendado	XAMPP 8.x (Windows) / LAMP Stack (Linux Ubuntu 20.04+)
Navegador	Google Chrome 100+, Firefox 100+, Edge 100+ (cualquier navegador moderno)
Zona Horaria	America/Tegucigalpa (configurable en parameters.env)

8.1 Instrucciones de Instalación

El procedimiento de instalación del sistema en un entorno local con XAMPP se realiza siguiendo los pasos indicados a continuación:

- Clonar o descomprimir el proyecto en la carpeta htdocs/inventario (XAMPP) o www/inventario (WAMP).
- Ejecutar el script docs/scripts/00_init_inventario.sql en MySQL mediante phpMyAdmin. Este script crea todas las tablas y datos iniciales necesarios.

- Copiar el archivo renameTo_parameters.env como parameters.env y actualizar los valores: DB_USER, DB_PSWD, DB_DATABASE y BASE_DIR=inventario.
- Ejecutar php composer.phar install dentro de la carpeta del proyecto para descargar las dependencias declaradas en composer.json.
- Abrir el navegador en: http://localhost/inventario/index.php?page=sec_login
- Iniciar sesión con las credenciales de prueba: propietario@inventario.com / Test1234\$

9. Limitaciones Actuales del Proyecto

En su estado actual de desarrollo correspondiente al primer parcial, el sistema presenta las siguientes limitaciones funcionales:

- Facturación y Punto de Venta: El sistema no cuenta con un módulo de emisión de facturas ni con un punto de venta (POS) integrado. Las salidas de inventario se registran manualmente como ajustes de stock sin generar documentos fiscales.
- Módulo de Clientes: No existe administración de perfiles ni base de datos de clientes compradores. Las tablas de soporte ya existen en la base de datos pero el módulo aún no está implementado.
- Reportería Gráfica: El dashboard actual no cuenta con visualización de datos mediante gráficos estadísticos de barras, pastel o líneas de tendencia.
- Generación de Archivos Exportables: El sistema no genera dinámicamente archivos .pdf o .xlsx desde el backend. La impresión de reportes depende del diálogo de impresión del navegador (window.print()) con estilos @media print.
- Validación Frontend en Tiempo Real: Los formularios de inventario no cuentan con validación JavaScript en el lado del cliente; las validaciones actualmente se realizan únicamente en el servidor (PHP).

- **Búsqueda con Filtros Avanzados:** Las vistas de listado no tienen implementada la búsqueda con filtros avanzados por múltiples criterios simultáneos en todas las pantallas.
- **Alerta Visual de Stock Bajo:** La lógica de alerta de stock bajo está definida en la base de datos pero aún no tiene una notificación visual activa implementada en el dashboard ejecutivo.

10. Posibles Mejoras Futuras

El diseño modular del sistema y la presencia de campos de extensión en la base de datos permiten proyectar las siguientes mejoras para versiones futuras del sistema:

- **CRUD Completo de Productos:** Completar la gestión de productos con carga de imagen del artículo y búsqueda interactiva por código de barras mediante lector físico o cámara web.
- **Kardex Interactivo y Exportable:** Implementar el Kardex completo con filtros por fecha, tipo de movimiento y producto, y funcionalidad de exportación directa a archivos Excel y PDF desde el servidor.
- **Dashboard con Gráficas de Control:** Desarrollar un Dashboard ejecutivo enriquecido con gráficas de stock crítico, top de productos más vendidos y evolución de movimientos recientes mediante una librería de visualización ligera.
- **Módulo de Facturación:** Agregar un módulo de emisión de facturas aprovechando los campos refTipo y refId ya presentes en la tabla movimientos_inventario, diseñados expresamente para este propósito.

- Integración de Lector de Código de Barras: Incorporar lectura de códigos de barras mediante la API de cámara web del navegador usando librerías como QuaggaJS o ZXing para agilizar el registro de entradas y salidas.
- Notificaciones por Correo Electrónico: Habilitar el envío de alertas automáticas por correo electrónico vía SMTP (ya configurado en parameters.env) cuando un producto alcance su nivel de stock mínimo o un lote esté próximo a vencer.
- API REST para Aplicación Móvil: Implementar una API REST que exponga los datos del inventario para su consumo desde una aplicación móvil futura, ampliando la accesibilidad del sistema.
- Soporte Multi-Sucursal: Agregar soporte para la gestión de inventarios en múltiples ubicaciones físicas, permitiendo consolidar y comparar el stock entre diferentes sucursales del negocio.
- Módulo de Clientes: Desarrollar la administración de perfiles de clientes recurrentes, habilitando la trazabilidad de ventas por cliente y la generación de reportes de comportamiento de compra.

Conclusiones

El Sistema de Control de Inventarios para Microempresas representa una solución tecnológica viable, accesible y directamente alineada con las necesidades reales del comercio informal hondureño. Su arquitectura MVC artesanal garantiza independencia de frameworks pesados, facilidad de mantenimiento y compatibilidad con entornos de hosting de bajo costo ampliamente disponibles en el país.

La implementación del algoritmo PEPS mediante el control de lotes por compra provee al propietario de la información contable necesaria para calcular con precisión los costos de adquisición y los márgenes de ganancia reales, superando la principal limitación del control manual empírico.

El sistema de roles tripartito (PRP, ADM, AUD) brinda una capa de seguridad robusta que permite delegar operaciones cotidianas sin comprometer la integridad del sistema, un requisito fundamental para las microempresas que contratan empleados de confianza.

Si bien el proyecto presenta limitaciones funcionales propias de una entrega de primer parcial, la base arquitectónica y el modelo de datos diseñados desde el inicio contemplan la incorporación futura de módulos como facturación, exportación de reportes y acceso móvil mediante API REST, lo que garantiza la escalabilidad del sistema más allá de su alcance actual.